Luiss
Libera Università Internazionale
degli Studi Sociali Guido Carli

Boolean Logic

Introduction to Computer Programming

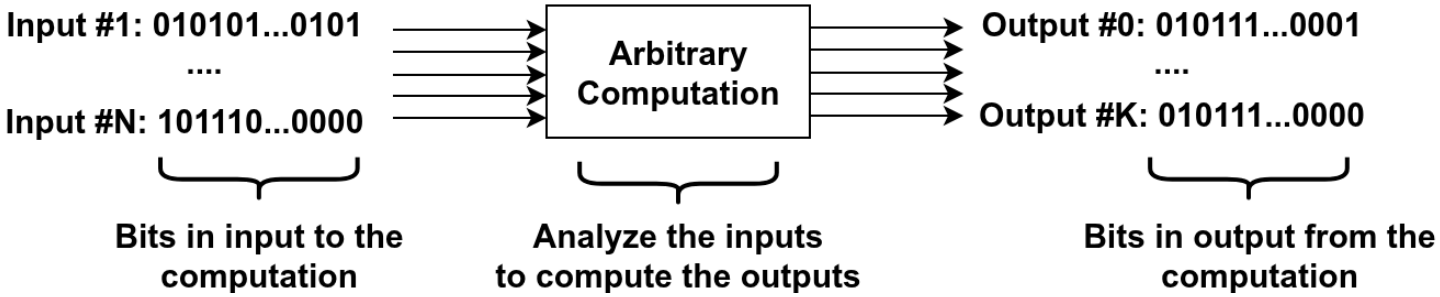
Management and Artificial Intelligence

LUISS

Bits Computations

Bits represent the data in a computer. ***But how can we perform meaningful computations over bits?***

At high level, we want to have:

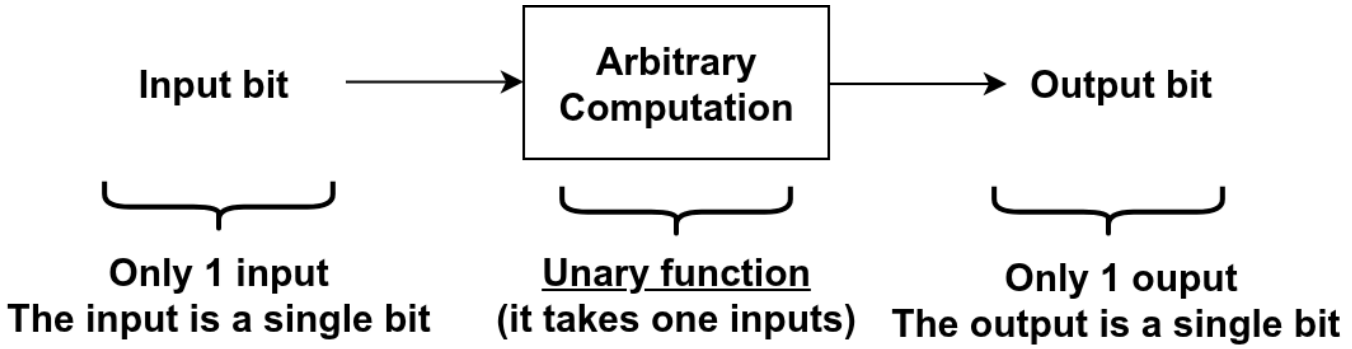


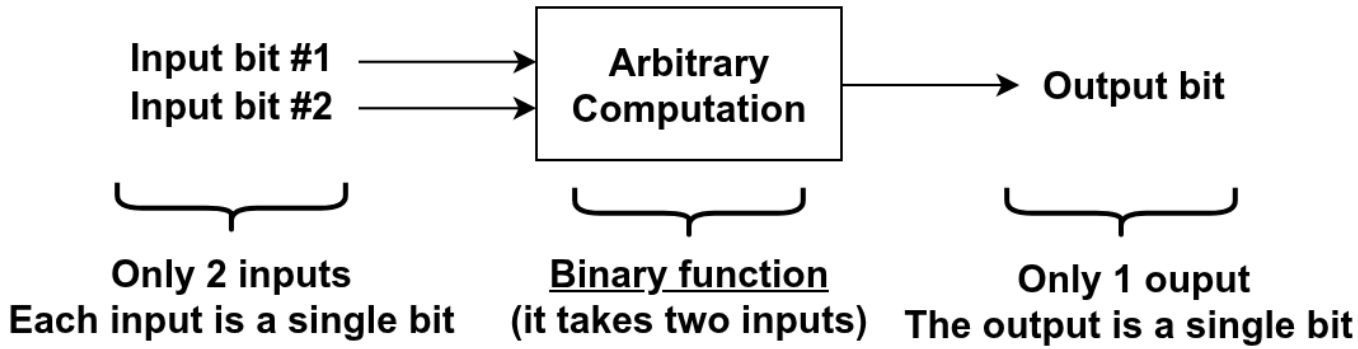
The computation can be anything we want:

- addition
- subtraction
- multiplication
- ...even complex operations such as apply a *filter* to a picture before posting it on social media!

Unary and Binary Functions

We need to start from simpler scenarios:





Inputs

The inputs of our computations will be binary values (0 and 1).

Since we want our computation to work on any possible assignment of values to the inputs, we need to consider ***ALL*** the possible assignments of values to the inputs:

- Possible input assignments for a unary function:

Input
0
1

- Possible input assignments for a binary function:

Input #1	Input #2
0	0
0	1
1	0
1	1

What if we ignore some input assignments?

Ignoring assignments is not a good idea:

- you are not defining what is expected output in some cases (the ignored input assignments)
- it leads your hardware/software to undefined behavior
- undefined behavior can lead to:
    - **bugs**: Mars Climate Orbiter (1999) from NASA exploded due to a software bug, leading to a $327.6 million loss.
    - **vulnerabilities**: Meltdown (2017) and Spectre (2018) from Intel CPUs may leak sensitive data; millions of devices are affected and required to be replaced.

Output

The output is a single bit (0 or 1). We need to specify the output value for each possible input assignment:

- Possible input assignments for a unary function:

Input	Output
0	*Either 0 or 1 depending on the function*
1	*Either 0 or 1 depending on the function*

- Possible input assignments for a binary function:

Input #1	Input #2	Output
0	0	*Either 0 or 1 depending on the function*
0	1	*Either 0 or 1 depending on the function*
1	0	*Either 0 or 1 depending on the function*
1	1	*Either 0 or 1 depending on the function*

How we determine the value of the output?

It depends on the computation!

For instance, if we want to perform multiplication:

Input #1	Input #2	Output
0	0	0
0	1	0
1	0	0
1	1	1

However, someone has spent a *bit* of time about thinking about to better formalize all of this and define which are the essential computation that we need (not only for a computer!):

The Mathematical Analysis of Logic by **George Boole** (1847)

Boolean Algebra

Boolean algebra:

- works on ***boolean*** values:
    - *true* (which corresponds to 1)
    - *false* (which corresponds to 0)
- defines some essential operators:
    - AND
    - OR
    - NOT
- shows that by combining these essential operators, using them as building blocks, we can create any complex computation

Modern computers are nothing more than efficient combinations of AND, OR, and NOT operations! They internally integrate billions of these operators to perform complex computations.

Boolean AND

The logical AND operator:

- is a binary operator that takes two inputs (e.g., denoted as variables *A* and *B*) and returns a single output (e.g., denoted as *A AND B*)
- is denoted by the symbol **AND**, $\wedge$, or **&&** or *
- is also called **conjunction**

The *truth table* for the AND operator is:

A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

INTUITION: the output is 1 (true) only when **both** inputs are 1 (true)

Boolean OR

The logical OR operator:

- is a binary operator that takes two inputs (e.g., denoted as variables *A* and *B*) and returns a single output (e.g., denoted as *A OR B*)
- is denoted by the symbol **OR**, $\vee$, or **||** or **+**
- is also called **disjunction**

The *truth table* for the OR operator is:

A	B	A OR B
0	0	0
0	1	1
1	0	1
1	1	1

INTUITION: the output is 1 (true) when **at least one** of the inputs is 1 (true)

Boolean NOT

The logical NOT operator:

- is a unary operator that takes a single input (e.g., denoted as variable *A*) and returns a single output (e.g., denoted as *NOT A*)
- is denoted by the symbol **NOT**, or ¬, or **!**, or **!**
- is also called **negation**

The *truth table* for the NOT operator is:

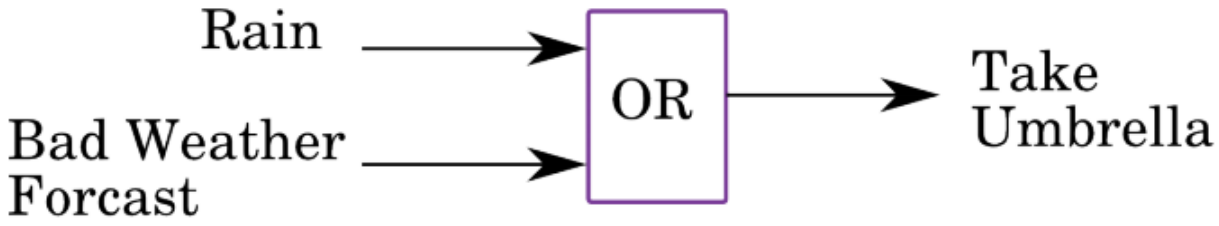
A	NOT A
0	1
1	0

INTUITION: the output is 1 (true) when the input is 0 (false), and vice versa

The operators in the real world

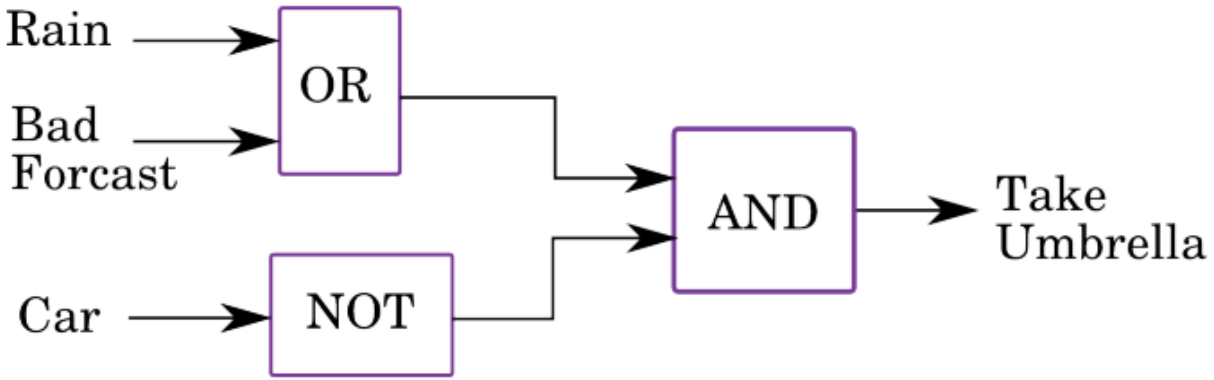
With a bit of fantasy, we can easily see that these essential operators can represent meaningful computation in the real world:

- Example of computation: *if either it rains or the weather forecast predicts rain, I take an umbrella*.



But the actual value begins when we start to combine (chain) them:

- Example of computation: *I take the umbrella if it rains or if the weather forecast predicts rain and I am not using the car*.



Other boolean operators

Other well-known boolean operators are XOR, NAND, and NOR:

A	B	A XOR B	A NAND B	A NOR B
0	0	0	1	1
0	1	1	1	0
1	0	1	1	0
1	1	0	0	0

These operators can be derived (implemented) from OR, AND, and NOT:

- XOR (exclusive OR):
    - equivalent to: OR(AND(A, NOT(B)), AND(NOT(A), B))
    - **INTUITION**: A XOR B is true if A and B are different
- NAND (not AND):
    - equivalent to NOT(AND(A, B))
    - **INTUITION**: A NAND B is true if A and B are not both true
- NOR (not OR):
    - equivalent to NOT(OR(A, B))
    - **INTUITION**: A NOR B is true if A and B are both false

Generic boolean formula

A generic boolean formula is a formula that can be evaluated to either True or False by applying boolean operators to its input boolean variables.

Some convention on the notation:

- A, B, C, etc. denote the boolean variables
- $\wedge$ is the boolean operator AND
- $\vee$ is the boolean operator OR
- $\neg$ is the boolean operator NOT
- Precedence of the operators (from highest to lowest):
    1. $\neg$
    2. $\wedge$
    3. $\vee$

Let us see some examples:

- $(A \wedge B) \vee (C \wedge D)$:

A	**B**	**C**	**D**	$(A \wedge B)$	$(C \wedge D)$	$(A \wedge B) \vee (C \wedge D)$
0	0	0	1	0	0	0
0	1	1	0	0	1	1
1	0	1	1	0	1	1
1	1	0	0	1	0	1

To make it easier for us, we have introduced in the truth table some intermediate computations ($(A \wedge B)$ and $(C \wedge D)$). Notice the parentheses in the formula are not needed since the formula $A \wedge B \vee C \wedge D$ would have the same truth table due to the operator precedence.

- $(\neg A \wedge B \wedge \neg C) \vee (A \wedge \neg B)$:

A	B	C	!A	!B	!C	(!A AND B)	(!A AND B AND !C)	(A AND !B)	(!A AND B AND !C) OR (A AND !B)
0	0	0	1	1	1	0	0	0	0
0	0	1	1	1	0	0	0	1	0
0	1	0	1	0	1	0	0	0	1
0	1	1	1	0	0	0	0	1	1
1	0	0	0	1	1	1	0	1	1
1	0	1	0	1	0	1	0	1	1
1	1	0	0	0	1	0	1	0	1
1	1	1	0	0	0	0	1	1	1

We have nine rows because we have three variables, each with two possible values (0 or 1).

Tautology

A Boolean expression is a tautology if it is always true, regardless of the values of the variables.

For example, these formulas are tautologies:

A	B	C	(A AND B) OR ! (A AND B)	!((A OR B) AND !(A OR B))	(A OR NOT A) OR (B OR C)	!(!(A OR B OR C)) OR !(A OR B OR C)
0	0	0	1	1	1	1
0	0	1	1	1	1	1
0	1	0	1	1	1	1
0	1	1	1	1	1	1
1	0	0	1	1	1	1
1	0	1	1	1	1	1
1	1	0	1	1	1	1
1	1	1	1	1	1	1

Contradiction

A Boolean expression is a contradiction if it is always false, regardless of the values of the variables.

For example, these formulas are contradictions:

A	B	C	(A and not A)	(A or B) and not(A or B)	(A and B and C) and not(A and B and C)	not(A or not A)
0	0	0	0	0	0	0
0	0	1	0	0	0	0
0	1	0	0	0	0	0
0	1	1	0	0	0	0
1	0	0	0	0	0	0
1	0	1	0	0	0	0
1	1	0	0	0	0	0
1	1	1	0	0	0	0

Equivalence

Two Boolean expressions are equivalent if they produce the same result for all possible values of the variables.

For example, these formulas are equivalent (compare 4th and 5th columns; compare 6th and 7th columns; compare 8th and 9th columns):

A	B	C	¬(A∧B)	¬A ∨ ¬B	¬(A∨B)	¬A ∧ ¬B	A ∨ (B ∧ C)	(A ∨ B) ∧ (A ∨ C)
0	0	0	1	1	1	1	0	0
0	0	1	1	1	1	1	0	0
0	1	0	1	1	0	0	0	0
0	1	1	1	1	0	0	1	1
1	0	0	1	1	0	0	1	1
1	0	1	1	1	0	0	1	1
1	1	0	0	0	0	0	1	1
1	1	1	0	0	0	0	1	1

Why do care about boolean formulas and their truth tables?

It turns out that we can:

- choose an arbitrary computation that we like: we define the truth table of the function as we like
- exploit several engineering techniques to turn the truth table into a boolean formula
- exploit other engineering techniques to *minimize* the boolean formula to an equivalent one with minimum number of operations
- implement the boolean formula in a circuit!

To do this for all expected computations that you need and you have build your first computer (or at least a part of it!)